

File permissions in Linux

How to read this document. ■ Dark boxes are the exact Linux commands I typed. ■ Screenshots are the output from the lab. Normal text is my own explanation. In permission strings, the colours show who each character applies to: **type** **user** **group** **other** .

Project description

In this project I am a security professional working with a research team, making sure that each user on the team is authorised with the appropriate permissions to keep the system safe. I must determine if the current permissions match the authorisation that should be given, and if they don't match, I need to modify the permissions to authorise the appropriate users and remove any access that isn't authorised.

Check file and directory details

First off I ran `ls -la ~/projects` to look inside the projects directory (`~/projects` is short for `/home/researcher2/projects`) and check what permissions each category of users had on the files and the subdirectory.

COMMAND I TYPED

```
researcher2@41703092394d:~$ ls -la ~/projects
```

`ls` lists the contents of a directory. The `-l` option shows the long format, which is what gives the permission string, the owner, the group, the size and the date for each item. The `-a` option shows all files, including hidden ones. Without `-a`, the hidden file `.project_x.txt` wouldn't have shown up at all.

LAB OUTPUT (SCREENSHOT)

```
researcher2@41703092394d:~$ ls -la ~/projects
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 12:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 13:26 ..
-rw--w---- 1 researcher2 research_team  46 Sep 27 12:21 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Sep 27 12:21 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Sep 27 12:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_t.txt
```

A dot at the start of a name means the file is hidden, and as you can see the only hidden file within the directory was `.project_x.txt`. The `.` and `..` entries also start with a dot, but they aren't files: `.` means the current directory and `..` means the parent directory, which here is `/home/researcher2`.

Describe the permissions string

Every line in the listing starts with a 10-character string, and this is what tells you who is authorised to do what with each file or directory. Once you know the rule, you can read any of them.

THE 10-CHARACTER RULE



Type is **d** for a directory or **-** for a file. Each set of three is **always** in the order **read, write, execute**.
A hyphen in a slot means that permission is **not** granted.

Who U, G and O are in this project

Letter	Who it means	In this project
U · user	The owner of the file	<code>researcher2</code> , who owns everything in <code>projects</code>
G · group	Members of the file's group	The <code>research_team</code> group
O · other	Every other user on the system	Anyone who isn't <code>researcher2</code> or in <code>research_team</code>

What r, w and x mean on a file compared with a directory

Permission	On a file	On a directory
r · read	Open and view the contents	List what's inside it
w · write	Change or overwrite the contents	Create, delete or rename files inside it
x · execute	Run it as a program or script	Enter it (<code>cd</code>) and access the files inside

Example: the drafts directory

Taking `drwx--x---` as an example: the **d** means drafts is a directory. The user set `rw` means `researcher2` can read, write and enter it. The group set `--x` means `research_team` can only enter it, so they can't list or change anything but can still get in. The other set `---` means everyone else has no access at all.

The chmod rule

To change a permission string I used `chmod`, which stands for change mode. It takes two arguments: what to change, then which file or directory to change it on.

Part	Options	Example from this project
Who	u g o a (all)	o -w targets other
Operator	+ add · - remove · = set exactly	o - w removes
Permission	r w x	o- w is write
Several at once	Separate with commas	g-w,g+r

The rule behind all of it: least privilege. Each user should only have the access they need to do their job and nothing more. Every change in this project either removes access that wasn't authorised or keeps access that was.

Change file permissions

The organisation doesn't allow other users to have write access to any files. Looking at `project_k.txt`, the string was `-rw-rw-rw-`. The `w` in the other set (the 9th character) meant "other" users had the ability to change and modify the project k file, which wasn't authorised. It was the only file with a `w` in that position.

So, to rectify that issue I had to change the permission for the other users:

COMMAND I TYPED

```
researcher2@41703092394d:~$ chmod o-w ~/projects/project_k.txt
researcher2@41703092394d:~$ ls -l ~/projects
```

`o-w` means take away (-) write (w) from other (o), on the file `~/projects/project_k.txt`. Then I listed the permissions in the projects directory so I could check that other users didn't have the write permission anymore, and they no longer do. The string is now `-rw-rw-r--`.

LAB OUTPUT (SCREENSHOT)

```
researcher2@41703092394d:~$ chmod o-w ~/projects/project_k.txt
researcher2@41703092394d:~$ ls -l ~/projects
total 20
drwx--x--- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Sep 27 12:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_t.txt
```

A change I didn't need to make: project_m.txt

Then I changed the permission of the `project_m.txt` file by removing the group's read permission.

COMMAND I TYPED

```
researcher2@41703092394d:~$ chmod g-r ~/projects/project_m.txt
researcher2@41703092394d:~$ ls -l projects
```

LAB OUTPUT (SCREENSHOT)

```
researcher2@41703092394d:~$ chmod g-r ~/projects/project_m.txt
researcher2@41703092394d:~$ ls -l projects
total 20
drwx--x--- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Sep 27 12:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_t.txt
researcher2@41703092394d:~$
```

Looking back, this wasn't actually needed. The string was `-rw-r-----`, so other users already had no write access and the file already followed the organisation's rules. The group never had write permission either, only read. By running `g-r` I took away access that research_team was legitimately authorised to have, which leaves the file at `-rw-----`. That's a problem too, because removing access people need can stop them doing their work. In a real environment I would put it back and only change what the policy requires:

FIX I WOULD RUN (NOT RUN IN THE LAB)

```
researcher2@41703092394d:~$ chmod g+r ~/projects/project_m.txt
```

Change file permissions on a hidden file

The research team archived `.project_x.txt`, which is why it's a hidden file. The requirement was that the user and group could read the file but nobody could write to it, which is `-r--r-----`. It started as `-rw--w-----`. It took four commands and two mistakes to get there.

First command. `chmod ug-w ~/projects/.project_x.txt` removed write permission from the user and group. The listing shows `-r-----`, so write access was gone, but the group still had no read access. The file still wasn't in the required state.

COMMAND I TYPED · 1

```
researcher2@41703092394d:~$ chmod ug-w ~/projects/.project_x.txt
researcher2@41703092394d:~$ ls -la projects
```

LAB OUTPUT (SCREENSHOT)

```
researcher2@41703092394d:~$ chmod ug-w ~/projects/.project_x.txt
researcher2@41703092394d:~$ ls -la projects
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 12:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 13:26 ..
-r----- 1 researcher2 research_team  46 Sep 27 12:21 .project_x.txt
drwx--x-- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Sep 27 12:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_t.txt
```

Second command. I ran `chmod g+w projects/.project_x.txt`, which gave the group write permission when what it needed was read. The listing shows `-r---w----`, the opposite of the goal, because write access was exactly what the task was meant to prevent. A single wrong letter reversed the security intent of the change.

COMMAND I TYPED · 2

```
researcher2@41703092394d:~$ chmod g+w projects/.project_x.txt
researcher2@41703092394d:~$ ls -la projects
```

LAB OUTPUT (SCREENSHOT)

```
researcher2@41703092394d:~$ chmod g+w projects/.project_x.txt
researcher2@41703092394d:~$ ls -la projects
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 12:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 13:26 ..
-r---w---- 1 researcher2 research_team  46 Sep 27 12:21 .project_x.txt
drwx--x-- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Sep 27 12:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_t.txt
```

Third command. To fix it I ran `chmod g-w,g+r`, but I misspelled the filename as `.porject_x.txt`. `chmod` returned `No such file or directory` and nothing changed. With hidden files the leading dot and the name both have to match exactly, and a small typo means the command fails to do its job unless you read the output.

Fourth command. I reran the same command with the correct spelling. It removed group write and added group read, and the final listing shows `-r--r-----`, the intended result.

COMMAND I TYPED · 3 AND 4

```
researcher2@41703092394d:~$ chmod g-w,g+r projects/.porject_x.txt
researcher2@41703092394d:~$ chmod g-w,g+r projects/.project_x.txt
researcher2@41703092394d:~$ ls -la projects
```

LAB OUTPUT (SCREENSHOT)

```
researcher2@41703092394d:~$ chmod g-w,g+r projects/.porject_x.txt
chmod: cannot access 'projects/.porject_x.txt': No such file or directory
researcher2@41703092394d:~$ chmod g-w,g+r projects/.project_x.txt
researcher2@41703092394d:~$ ls -la projects
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 12:21 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 27 13:26 ..
-r--r----- 1 researcher2 research_team  46 Sep 27 12:21 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_k.txt
-rw----- 1 researcher2 research_team  46 Sep 27 12:21 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 27 12:21 project_t.txt
researcher2@41703092394d:~$
```

What this shows. The whole task could have been done in one command: `chmod ug=r projects/.project_x.txt`. The `=` operator sets permissions exactly, whereas `+` and `-` only adjust what is already there, so you have to know the starting state and track every change.

Change directory permissions

Finally I changed the permissions on the `~/projects/drafts` directory. The files and directories in projects belong to researcher2, and researcher2 was the only one authorised to access drafts and nobody else.

BEFORE

```
drwx--x--- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
```

The string was `drwx--x---`, which means the group had execute permission. On a directory, execute is what lets you enter it and access the files inside, so research_team could still get into drafts even though they couldn't list what was in it. That's why I removed execute rather than read: the group never had read, and `x` was the only permission giving them access. So same as always I changed the permission for the group:

COMMAND I TYPED

```
researcher2@41703092394d:~$ chmod g-x projects/drafts
researcher2@41703092394d:~$ ls -la projects
```

AFTER

```
drwx----- 2 researcher2 research_team 4096 Sep 27 12:21 drafts
```

Now researcher2 is the only user within the system that has access to drafts: `drwx-----`. Other users already had no permissions, so nothing needed to change there.

Before and after

Item	Before	Command	After	Why
project_k.txt	-rw-rw-rw-	o-w	-rw-rw-r--	Others can't write
.project_x.txt	-rw--w----	ug-w , g-w, g+r	-r--r------	Archived, read-only
drafts	drwx--x---	g-x	drwx-----	Owner only
project_m.txt	-rw-r-----	g-r	-rw-----	Not needed, revert
project_r.txt , project_t.txt	-rw-rw-r--	none	-rw-rw-r--	Already compliant

Summary

To sum up, my project was to examine the file and directory permissions in the projects directory, then change them so each user is rightfully authorised to specific files and directories. I used `ls -la` to check the permissions, including the hidden file, and read the 10-character strings to find what didn't match the organisation's rules. I then used `chmod` to remove write access for other users on `project_k.txt`, make the archived `.project_x.txt` read-only for the user and group, and restrict `drafts` so only `researcher2` can access it. Along the way I learned to check the output after every command, because a single wrong letter or typo can undo the whole point of the change.